Running Scenario for the ERASMUS session in Antwerp

Before you start your presentation

- Open a terminal at ~/Projects/2025-ERASMUS-Antwerp-Sorensen/JuliaIntro
- Start another terminal at ~/Projects/2025-ERASMUS-Antwerp-Sorensen/JuliaIntro/juliasrc and launch jupyter lab
- Start another terminal at ~/Projects/2025-ERASMUS-Antwerp-Sorensen/JuliaIntro/juliasrc and launch `code`
- Open a terminator at ~/Projects/2025-ERASMUS-Antwerp-Sorensen/JuliaIntro/juliasrc for the REPL
- Start a browser with [http://julialang.org]
- Start a tab with [https://www.kaggle.com/datasets/sudalairajkumar/daily-temperature-of-major-cities]
- Start a tab with [https://juliapackages.com]
- Start code (with testmean.jl)
- Start evince with full screen presentation

Slides

Slide #1

- Thank people for the invitation
- Thank my team for support, discussions, corrections, etc

Slide #2

- Presentation

Slide #3

- Explain where julia is used in class and on which topics

Slide #4

- Tell how many languages studied/used with which success (not counting scripting languages)
- Show source code and explain how easy

Slide #5

- There are many reasons behind Julia's rise to popularity. While Python and R reign supreme as the most widely used programming languages in data science, Julia has been downloaded over 35 million times. The reasons behind its growth are thoroughly outlined in this article, but here's a summary of Julia's features and advantages:
- *Speed:* One of Julia's main founding principles is that it needs to be as general as Python, as statistics-friendly as R, but as fast as C. This enables faster data workflows, especially for scientific computing.
- *Syntax:* Another way Julia shines is its syntax. Despite its speed, Julia's syntax is dynamically-typed, making it closer to R and Python than more complex programming languages. This means it's much more accessible and easy to learn than other high-performance programming languages.
- *Multiple dispatch:* Without getting too technical, another advantage of Julia is multiple dispatch, which refers to Julia functions' ability to behave differently based on the types of arguments it is given. This makes it a flexible language to work with.
- Learn Julia Now And Thank Your Younger Self in the Future

Slide #6

- Show slide and switch to real web page

Show **BROWSER** http://julialang.org

- Repeat the main features (Fast, Dynamic, Open Source, Easy to learn)
- Show the download page and tell people that everything is well explained
- Show Documentation with for example the `mean` function
- Show the Learn page and tell them that we are here for that

Slide #7

- Show slide and switch to REPL in terminator

Show REPL in Terminator

- start the REPL (Read Evaluation Print Loop), execute a piece of code

```
using Statistics
A = rand(0:999,10) # add ; to avoid output
mean(A)
```

- Show **CODE**: show that this could be done in an editor like code
- make a .jl file that you can execute from the **REPL**.
- In the **REPL**, you can show that ; give you prompt command (;ls) show as well that you can access help with ? (?if)
- show that the same file can be executed through the **command line**
- start **jupyter** and show that the same code can be executed here
- come back to the REPL and add the following piece of code:

```
println(A)
println(A')
\lambda = 3
\mu = \lambda + 7
\mu * A
using LinearAlgebra
B = rand(-1:3,3,3)
det(B)
```

Slide #8

- So you write f(x), then call f(1.0), it'll compile something specifically for Float64. If none of the types change in that function (called type-stability), then everything can be statically-typed, so Julia compiles a version of the function where everything is statically typed, and thus you get the speed of a statically-typed language after the first call which just compiles. So basically it is multiple dispatch, i.e. the ability to treat functions as different depending on the types that you call it with, that lets Julia compile statically-typed functions for each concrete type you throw in there.

Show **jupyter** notebook "Julia-is-fast.pynb"

At least **julia is as fast as Python+numpy** for numerical computations. It has been designed for speed.

Slide #9

Show **REPL** with the following code

```
using DataFrames, CSV
# download the file from
# https://www.kaggle.com/datasets/sudalairajkumar/daily-temperature-of-major-cities

@time df = DataFrame(CSV.File("city_temperature.csv"))
### as fast/faster than Pandas library from Python
df.AvgTemperature
using Statistics
mean(df.AvgTemperature)
temperatures = df.AvgTemperature

function toCelsius(x)
    y = (x-32)*5/9
    return y
end

### Say here that julia is very portable since no risk of loosing tabs and spaces. There is always an en

toCelsius(mean(temperatures))

toC(x) = (x-32)*5/9

Ctemp = toC.(temperatures)
mean(toC.(temperatures))
mean(toC.(temperatures)) |> floor
mean(toC.(temperatures)) |> x -> round(x,digits=2)
```

```
count(i->i>=43, Ctemp)
v = filter(i->i .\ge 43, Ctemp)

using StatsBase
sort(countmap(v))
```

Slide #10

- Show the slide

BROWSER

- Show the **BROWSER** with https://juliapackages.com
- Click on one category
- Click on Flux.jl package

REPL

- Open the **REPL**
- Show the package manager `]`
- Type `status`
- Add the Example package with `add Example`
- Type `status`
- Back to the REPL, type `hello("Marc")`

CODE

- Open tab `testgraphs.jl`
- Run the code line by line (to have the graph displayed)

REPL

- Copy and paste this piece of code

```
using PlotlyJS

unique(df[df.City .== "Paris",:].Year)
list_years = [1995, 2005, 2015]
Tavg = [mean(filter(x -> x.City == "Paris" && x.Year == year && x.Month == month, df)[:,"AvgTemperature"
    list_traces = AbstractTrace[]
for i in eachindex(list_years)
    trace = scatter(x=[1:12;], y=Tavg[i,:], name=list_years[i])
    push!(list_traces, trace)
end
plot(list_traces)
```

JUPYTER

- run the `cutting-stock` notebook

Slide #11

- Conclusion Thank the younger self in the future